Assignment No: 7

Real-Time Application Monitoring Using Prometheus

Aim

To configure Prometheus to collect and monitor real-time performance metrics of the Student Task Manager application.

Output

1. Install client and test /metrics locally

```
aditya@aditya:~/devops-task-manager$ npm install prom-client
added 3 packages in 2s
aditya@aditya:~/devops-task-manager$ npm start
Student Task Manager running on port 3000
http://localhost:3000/metrics
# HELP task_manager_http_requests_total Total HTTP requests received
# TYPE task_manager_http_requests_total counter
task_manager_http_requests_total 3
# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 68157440
```

2. Rebuild and redeploy application

```
aditya@aditya:~/devops-task-manager$ docker build -t task-manager:2.0 .
Successfully built Docker Image task-manager:2.0
aditya@aditya:~/devops-task-manager$ minikube image load task-manager:2.0
aditya@aditya:~/devops-task-manager$ kubectl apply -f deployment.yaml
deployment.apps/task-manager-deployment configured
aditya@aditya:~/devops-task-manager$ kubectl get pods
task-manager-deployment-yyyyy-aaaaa   1/1   Running   0   15s
task-manager-deployment-yyyyy-bbbbb   1/1   Running   0   15s
```

3. Prometheus configuration and deployment

```
aditya@aditya:~/devops-task-manager$ kubectl create configmap prometheus-config --from-file=prometheus.yml
configmap/prometheus-config created
aditya@aditya:~/devops-task-manager$ kubectl apply -f prometheus-deployment.yaml
deployment.apps/prometheus-deployment created
aditya@aditya:~/devops-task-manager$ kubectl apply -f prometheus-service.yaml
service/prometheus-service created
aditya@aditya:~/devops-task-manager$ kubectl get pods
prometheus-deployment-zzzzz-aaaaa   1/1   Running   0   30s
```

4. Access Prometheus

```
aditya@aditya:~/devops-task-manager$ minikube service prometheus-service
* Opening service default/prometheus-service in default browser...
```

5. Status > Targets

```
Job: task-manager
Endpoint: http://task-manager-service:3000/metrics
State: UP
Prometheus Target Status
Job: task-manager
State: UP
```

6. PromQL queries

```
> up{job="task-manager"}
up{job="task-manager"}   1

> task_manager_http_requests_total
task_manager_http_requests_total{job="task-manager"}   45

> rate(task_manager_http_requests_total[1m])
{job="task-manager"}   0.35

> process_resident_memory_bytes
process_resident_memory_bytes{job="task-manager"}   68157440
```

7. Final result

```
HTTP Request Counter:
task_manager_http_requests_total = 45
Application Availability:
up = 1
Process Memory:
process_resident_memory_bytes = 68157440
Prometheus successfully collecting application metrics.
```

Conclusion

Prometheus was configured to scrape the /metrics endpoint every 5 seconds, and key application metrics were observed using PromQL.